Sentinel — In-Database Real-Time Fraud Detection System

Database Management Systems (CS/AI/AIM/DS 202) — Mini Project Report

Institution: ICFAI Foundation for Higher Education, Faculty of Science & Technology, Hyderabad

Programme: B.Tech CSE (AI & ML), Semester III, A.Y. 2026-27

Team: Sai Haresh Anand S · Seshivardhini

Repository: https://github.com/SAI-HARISH2007/sentinel-sql-progress **Database:** MySQL 8.4 (Docker)

Abstract

Fraud checks usually live in application code: hard-coded thresholds, a gap between commit and check, and a separate copy of the data. Sentinel moves the check *into the database*. A trigger on `INSERT INTO transactions` calls a stored procedure that scores the new row against six rules (high amount, off-hours, new device, velocity, impossible travel, merchant risk), each measured against that account's own history, and writes an alert with human-readable reasons. Thresholds live in a table, so tuning is an `UPDATE`.

The report is organised **week by week**, following the course's lab sequence. Each page was added in the week the matching lab was done, and every query result shown was captured by running the repository's own scripts on the finished database.

Week	Lab topic	Adds to the project
1	Introduction to MySQL & database design (ER)	Design before you build.
2	DDL: CREATE, ALTER, DROP, TRUNCATE, RENAME	Create all 13 tables with columns and data types only.
3	Constraints: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, DEFAULT, CHECK	Turn the bare tables into a self-defending schema.
4	DML: INSERT, UPDATE, DELETE, SELECT	Put data in.
5	Functions: number, aggregate, string, date	Use every function family on real data, and use them to build the first three pieces of fraud logic by hand: the z-score (Rule 1), the hour of day (Rule 2) and the distance and speed between cities (Rule 5).
6	Operators: arithmetic, comparison, logical, special, set	Every operator family on real data, ending in a deliberately naive "first-cut fraud screen" built from fixed thresholds, which shows *why* the later weeks compare each account with its own history.
7	Joins: inner, outer, natural, self	Combine tables.
8	GROUP BY, HAVING, ORDER BY — and subqueries	Aggregate per group and filter groups.
9	Views and indexes (plus DCL: roles, GRANT, REVOKE)	Package the eight weeks of queries as named, reusable views, speed up the access paths with measured indexes, and lock the data down with roles.
10	Stored procedures, triggers — and transactions	The finale: the same six rules, now as one stored procedure that a trigger runs on every INSERT.

Week	Lab topic	Adds to the project
11	Testing, documentation and the demo	Prove it works, write it up, rehearse it.

Week 1 — Introduction to MySQL & database design (ER)

Project README · Week 2 →

Lab: Handout lab list **#1 — Introduction to MySQL and ER Diagram** · lab manual **Experiment 0/1 — Introduction to SQL & MySQL and Database Design Theory lectures this lines up with:** 1–9 (introduction to databases, ER modelling, ER → relational design)

Design before you build. You create the empty `sentinel` database and, more importantly, write down the problem, the requirements, the ER diagram and the rules that turn that diagram into tables. No table exists yet.

What was added

File	What it is
`sql/01_create_database.sql`	`DROP/CREATE DATABASE sentinel`, utf8mb4. Every later week starts from this.
`docs/problem-statement.md`	Handout steps 1-4: title, problem, objectives, functional + non-functional requirements, the six fraud signals.
`docs/er-diagram.md` · `er-diagram.dbml`	The ER diagram (Mermaid, renders on GitHub) and DBML for dbdiagram.io. **Generated from the real schema**, never hand-edited.
`docs/er-to-relational.md`	Which ER-to-tables rule produced which table (strong entity, 1:N, M:N, 1:1, participation, derived attribute).
`demo/01_mysql_basics.sql`	`SHOW DATABASES`, `USE`, `SHOW TABLES`, `VERSION()`: the commands in the lab manual's intro page.

Files in `sql/` are **loaded by the build** (`scripts/build.sh`). Files in `queries/`, `demo/` and `lab-practice/` are run by hand and are checked by `scripts/check.sh`.

What you should see

- `SHOW DATABASES` lists `sentinel`; `SHOW TABLES` is empty (tables arrive in Week 2).
- Character set `utf8mb4`, collation `utf8mb4_unicode_ci`.

Concepts you must be able to explain (viva)

- **DBMS vs file system.** A DBMS adds integrity rules, concurrent access control, security and data independence that plain files lack.
- **The five parts of SQL.** DDL (structure) · DML (change data) · DRL (SELECT) · TCL (COMMIT/ROLLBACK) · DCL (GRANT/REVOKE). The lab manual lists exactly these five.
- **ER vocabulary.** Entity, entity set, attribute (simple/composite/multivalued/derived), relationship, cardinality (1:1, 1:N, M:N), participation (total/partial), weak entity.
- **ER → relational.** Read `docs/er-to-relational.md` twice. Each row is a Part B/C exam answer.
- **Why surrogate keys.** `email`, `fingerprint_hash` are candidate keys; a small `INT` id is the primary key because it is compact in every foreign key and index.

More questions with full answers: `week-11-final-polish/docs/viva_questions.md`. Syntax for the closed-book lab: `docs/LAB_EXAM_CHEATSHEET.md`.

Handout step covered

Handout steps 1-5 (title, problem, objectives, requirements, ER model).

Results (captured from the running database)

Evidence: Database and character set

Script: `week-01-intro-er-design/demo/01_mysql_basics.sql`

```
+--------------+
| mysql_version |
+--------------+
| 8.4.11        |
+--------------+
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sentinel           |
| sys                |
+--------------------+
+-----------------+
| current_database |
+-----------------+
| sentinel         |
+-----------------+
+------------------------+
| Tables_in_sentinel     |
+------------------------+
| account_devices        |
| accounts               |
| alerts                 |
| audit_log              |
| behaviour_baselines    |
| devices                |
| locations              |
| login_history          |
... (22 more lines)
```

Week 2 — DDL: CREATE, ALTER, DROP, TRUNCATE, RENAME

Lab: Handout lab list **#2 — Data Definition Language** (CREATE, ALTER, DROP, TRUNCATE, RENAME) · lab manual **"Experiment 2: DDL COMMANDS"** (its index calls it Experiment 1) **Theory lectures this lines up with:** 12–17 (basic SQL: DDL)

Create all 13 tables with columns and data types only. Four things are created deliberately wrong so the ALTER / RENAME commands have real work to do. No keys, no defaults: that is next week.

What was added

File	What it is
`sql/01_create_tables.sql`	13 `CREATE TABLE` statements: columns and types, nothing else.
`sql/02_alter_tables.sql`	`ADD COLUMN`, `RENAME COLUMN`, `MODIFY`, `DROP COLUMN`, `RENAME TABLE`, each fixing one deliberate flaw.
`demo/ddl_tour.sql`	Every DDL command from the manual on a throw-away copy (`CREATE TABLE … LIKE`), cleaned up afterwards.
`lab-practice/ddl_employee.sql`	**The lab manual's 9-question DDL assignment (EMPLOYEE table), solved in MySQL**, in its own `lab_practice` database.

Files in `sql/` are **loaded by the build** (`scripts/build.sh`). Files in `queries/`, `demo/` and `lab-practice/` are run by hand and are checked by `scripts/check.sh`.

What you should see

- 13 tables (`SHOW TABLES`). `DESCRIBE users` has `date_of_birth` right after `phone`, and the `Key` column is **empty** for every table.
- `devices.fingerprint_hash` is `varchar(64)`; `merchants.merchant_name` is `varchar(200)`; `fraud_alerts` is now called `alerts` and has no `notes` column.

Concepts you must be able to explain (viva)

- **Data types.** `INT`, `BIGINT`, `DECIMAL(p,s)` (exact: use for money, never FLOAT), `CHAR` vs `VARCHAR`, `TEXT`, `ENUM`, `JSON`, `TIMESTAMP`, `BOOLEAN` (= `TINYINT(1)`).
- **ALTER TABLE.** `ADD`, `MODIFY` (change type/width), `RENAME COLUMN`, `DROP COLUMN`.
- **DELETE vs TRUNCATE vs DROP.** DELETE: chosen rows, can roll back, fires triggers. TRUNCATE: all rows, DDL, resets AUTO_INCREMENT, no rollback. DROP: rows *and* table.
- **DDL commits implicitly.** You cannot ROLLBACK a CREATE/ALTER/DROP/TRUNCATE.
- **Why bare tables first?.** A table without keys is legal but unsafe: it would accept duplicate users and orphan transactions. Week 3 shows the fix.

More questions with full answers: `week-11-final-polish/docs/viva_questions.md`. Syntax for the closed-book lab: `docs/LAB_EXAM_CHEATSHEET.md`.

Handout step covered

Handout step 7 (SQL implementation: DDL).

Results (captured from the running database)

Evidence: Every DDL command on a throw-away table

Script: `week-02-ddl/demo/ddl_tour.sql`

```
+---------------+-------------+------+-----+-------------------+-------------------+
| Field         | Type        | Null | Key | Default           | Extra             |
+---------------+-------------+------+-----+-------------------+-------------------+
| user_id       | int         | NO   | PRI | NULL              | auto_increment    |
| full_name     | varchar(150) | NO   |     | NULL              |                   |
| email         | varchar(255) | NO   | UNI | NULL              |                   |
| phone         | varchar(20)  | YES  |     | NULL              |                   |
| date_of_birth | date        | YES  |     | NULL              |                   |
| created_at    | timestamp   | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+---------------+-------------+------+-----+-------------------+-------------------+
+---------------+-------------+------+-----+-------------------+-------------------+
| Field         | Type        | Null | Key | Default           | Extra             |
+---------------+-------------+------+-----+-------------------+-------------------+
| user_id       | int         | NO   | PRI | NULL              | auto_increment    |
| full_name     | varchar(150) | NO   |     | NULL              |                   |
| email         | varchar(255) | NO   | UNI | NULL              |                   |
| phone         | varchar(20)  | YES  |     | NULL              |                   |
| date_of_birth | date        | YES  |     | NULL              |                   |
| created_at    | timestamp   | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+---------------+-------------+------+-----+-------------------+-------------------+
+------------+-------------------
```

Week 3 — Constraints: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, DEFAULT, CHECK

Lab: Handout lab list **#3 — Constraints** (PRIMARY KEY, FOREIGN KEY, UNIQUE, NULL, NOT NULL, DEFAULT, CHECK, DROP) · lab manual "Constraints" **Theory lectures this lines up with:** 12–17 (keys, constraints); 23–25 (normalization) for `docs/normalization.md`

Turn the bare tables into a self-defending schema. Every rule is added with `ALTER TABLE` in the order the manual lists them, so you can see each constraint appear.

What was added

File	What it is
`sql/01_constraints.sql`	Primary keys (+ AUTO_INCREMENT), NOT NULL, UNIQUE, DEFAULT, CHECK, FOREIGN KEY with ON DELETE rules.
`demo/constraint_violations.sql`	Makes every constraint fire: six statements that **must fail**, with the exact MySQL error code, plus one CASCADE that succeeds.
`demo/drop_and_readd_constraints.sql`	The manual's *DROP* part: drop and restore a CHECK, a FOREIGN KEY, a UNIQUE and a DEFAULT.
`docs/normalization.md`	The 1NF / 2NF / 3NF / BCNF argument, table by table.

Files in `sql/` are **loaded by the build** (`scripts/build.sh`). Files in `queries/`, `demo/` and `lab-practice/` are run by hand and are checked by `scripts/check.sh`.

What you should see

- 13 PRIMARY KEY, 13 FOREIGN KEY, 7 UNIQUE, 4 CHECK constraints.
- The violation demo prints, in order: **1062** (dup primary key), **1062** (dup unique), **1048** (NULL), **1452** (missing parent), **3819** (CHECK), **1451** (RESTRICT). The CASCADE then shows `accounts_left_after_cascade = 0`.

Concepts you must be able to explain (viva)

- **Primary key vs unique.** One PK per table, never NULL. Any number of UNIQUE keys. Both are candidate keys.
- **Composite key.** `account_devices` is identified by the *pair* `(account_id, device_id)`.
- **Foreign key + ON DELETE.** CASCADE deletes children; RESTRICT blocks the delete. Choose per relationship: see the table in the SQL file.
- **CHECK.** Enforced from MySQL 8.0.16; older versions silently ignore it.
- **NULL.** "Unknown", not zero or empty. A column never given NOT NULL is deliberately optional.
- **Order matters.** Keys first, foreign keys last: an FK needs a PK/UNIQUE key to point at.

More questions with full answers: `week-11-final-polish/docs/viva_questions.md`. Syntax for the closed-book lab: `docs/LAB_EXAM_CHEATSHEET.md`.

Handout step covered

Handout steps 6-7 (schema design, normalisation; SQL implementation).

Results (captured from the running database)

Evidence: Constraints firing (errors are the expected result)

Script: `week-03-constraints/demo/constraint_violations.sql`

```
+------------+--------------+---------+--------+
| account_id | account_type | balance | status |
+------------+--------------+---------+--------+
|      99001 | savings      |    0.00 | active |
+------------+--------------+---------+--------+
+----------------------------+
| accounts_left_after_cascade |
+----------------------------+
|                          0 |
+----------------------------+
ERROR 1062 (23000) at line 11: Duplicate entry '99001' for key 'merchant_categories.PRIMARY'
ERROR 1062 (23000) at line 14: Duplicate entry 'Demo category' for key 'merchant_categories.category_name'
ERROR 1048 (23000) at line 17: Column 'full_name' cannot be null
ERROR 1452 (23000) at line 20: Cannot add or update a child row: a foreign key constraint fails (`sentinel`.`accounts`, CONSTRAINT `fk_accounts_user` FOREIGN KEY (`us
ERROR 3819 (HY000) at line 23: Check constraint 'chk_txn_amount' is violated.
ERROR 1451 (23000) at line 29: Cannot delete or update a parent row: a foreign key constraint fails (`sentinel`.`merchants`, CONSTRAINT `fk_merchants_category` FOREI(
```

Week 4 — DML: INSERT, UPDATE, DELETE, SELECT

Lab: Handout lab list **#4 — Data Manipulation Language** (SELECT, INSERT, UPDATE, DELETE) · lab manual "Experiment 3: DML COMMANDS" **Theory lectures this lines up with:** 12–17 (basic SQL: DML)

Put data in. The detector's thresholds go into `rule_config`, and a deterministic generator loads 500 users, 12,067 transactions and five planted fraud personas. From now on every week queries the same data.

What was added

File	What it is
`sql/10_rule_config.sql`	8 tuning rows (z-score limit, velocity limit, …). The detector will read these; changing them is an UPDATE.
`sql/20_seed_data.sql`	The generated data set. **Committed**, so nobody needs Python to build the project.
`seed/generate.py`	The generator (Faker, fixed random seed). Re-run only if you want different data.
`demo/dml_tour.sql`	Single-row, multi-row and `INSERT … SELECT`; UPDATE; DELETE; a CASCADE; self-cleaning.
`lab-practice/dml_employee.sql`	**The manual's 9-question DML assignment (EMPLOYEE table), solved.**

Files in `sql/` are **loaded by the build** (`scripts/build.sh`). Files in `queries/`, `demo/` and `lab-practice/` are run by hand and are checked by `scripts/check.sh`.

What you should see

- users 500 · accounts 597 · merchants 50 · merchant_categories 10 · locations 30 · devices 201 · account_devices 1,780 · login_history 200 · **transactions 12,067** · rule_config 8.
- Personas, all on rows 12041-12067: **traveller** 12041/12042 (Delhi then Chennai, 10 min apart) · **night_owl** 12043 (02:30) · **burst** 12044-12051 (8 payments in 3 min) · **new_device_big** 12052 (₹95,000 from device 999) · **bad_merchant** 12053-12067 (15 large payments at merchant 50).

Concepts you must be able to explain (viva)

- **Three INSERT forms.** One row · many rows in one statement · `INSERT … SELECT`.
- **UPDATE / DELETE need WHERE.** Without it they change *every* row. This is the classic exam trap.
- **Defaults in action.** `INSERT INTO accounts (account_id, user_id)` gives savings / 0.00 / active, because of Week 3.
- **CASCADE in action.** Deleting user 90001 removes their account (demo).
- **Config as data.** Thresholds in a table mean tuning needs no code change and no redeploy.
- **Why `SET autocommit = 0` in the seed.** One commit for 15,000 inserts instead of 15,000 commits: far faster.

More questions with full answers: `week-11-final-polish/docs/viva_questions.md`. Syntax for the closed-book lab: `docs/LAB_EXAM_CHEATSHEET.md`.

Handout step covered

Handout step 7 (SQL implementation: DML) and step 11 (test data).

Results (captured from the running database)

Evidence: INSERT / UPDATE / DELETE tour

Script: `week-04-dml/demo/dml_tour.sql`

```
+---------+-----------+------------------+--------------+--------------+---------------------+
| user_id | full_name  | email             | phone         | date_of_birth | created_at          |
+---------+-----------+------------------+--------------+--------------+---------------------+
|   90001 | Demo One   | demo1@example.com | +919000000001 | 2005-05-05    | 2026-10-08 02:12:15 |
|   90002 | Demo Two   | demo2@example.com | NULL          | NULL          | 2026-10-08 02:12:15 |
|   90003 | Demo Three | demo3@example.com | NULL          | NULL          | 2026-10-08 02:12:15 |
+---------+-----------+------------------+--------------+--------------+---------------------+
+------------+--------------+---------+--------+---------------------+
| account_id | account_type | balance | status | created_at          |
+------------+--------------+---------+--------+---------------------+
|      90001 | savings      |    0.00 | active | 2026-10-08 02:12:15 |
+------------+--------------+---------+--------+---------------------+
+-------------+---------------------------+--------+
| merchant_id | merchant_name              | city   |
+-------------+---------------------------+--------+
|          16 | Miller, Davis and Anderson | Mumbai |
|          14 | Morse PLC                  | Mumbai |
+-------------+---------------------------+--------+
+---------+
| status  |
+---------+
| failed  |
| success |
| pending |
+---------+
+--------+------------+----------+---------------------+
| txn_id | account_id | amount   | txn_time            |
+--------+------------+----------+---------------------+
|  12052 |          1 | 95000.00 | 2026-08-18 11:00:00 |
|  12059 |        285 | 69888.23 | 2026-08-19 18:00:00 |
|  12053 |        117 | 66536.48 | 2026-08-19 12:00:00 |
|  12062 |        165 | 62629.96 | 2026-08-19 21:00:00 |
|  12061 |        158 | 61521.54 | 2026-08-19 20:00:00 |
+--------+------------+----------+---------------------+
+------------+---------+
| account_id | balance |
+------------+---------+
|      90001 | 5000.00 |
+------------+---------+
+-----------+-----------------+---------------------+
| rule_name | threshold_value | updated_at          |
+-----------+-----------------+---------------------+
| zscore    |          3.0000 | 2026-10-08 02:06:16 |
+-----------+-----------------+---------------------+
+-----------+-----------------+---------------------+
| rule_name | threshold_value | updated_at          |
+-----------+-----------------+---------------------+
| zscore    |          2.5000 | 2026-10-08 02:12:15 |
+-----------+-----------------+---------------------+
+-----------------+
| demo_users_left |
+-----------------+
|               2 |
+-----------------+
+--------------------+
| demo_accounts_left |
+--------------------+
|                  0 |
+--------------------+
```

Week 5 — Functions: number, aggregate, string, date

Lab: Handout lab list **#5 — Functions** (NUMBER, AGGREGATE, CHARACTER, DATE) · lab manual "Implementation of different types of function" **Theory lectures this lines up with:** 12–17 (functions)

Use every function family on real data, and use them to build the first three pieces of fraud logic by hand: the z-score (Rule 1), the hour of day (Rule 2) and the distance and speed between cities (Rule 5).

What was added

File	What it is
`queries/01_number_functions.sql`	ROUND, CEIL, FLOOR, TRUNCATE, MOD, ABS, SQRT, POWER, GREATEST/LEAST, and the Haversine formula from sines and cosines.
`queries/02_aggregate_functions.sql`	COUNT, SUM, AVG, MIN, MAX, STDDEV_SAMP, VARIANCE; `COUNT(*)` vs `COUNT(col)`.
`queries/03_string_functions.sql`	UPPER, CONCAT, LEFT/RIGHT, SUBSTRING, INSTR, REPLACE, LPAD, TRIM; masking emails and phones.
`queries/04_date_functions.sql`	NOW, DATE, HOUR, DAYNAME, DATE_FORMAT, DATE_ADD, DATEDIFF, TIMESTAMPDIFF.
`queries/05_rule1_high_amount.sql`	**Rule 1 for one account**, with user variables.
`queries/06_rule2_off_hours_preview.sql`	Rule 2 preview using `HOUR()`.
`queries/07_rule5_impossible_travel_by_hand.sql`	**Rule 5 for the traveller persona.**

Files in `sql/` are **loaded by the build** (`scripts/build.sh`). Files in `queries/`, `demo/` and `lab-practice/` are run by hand and are checked by `scripts/check.sh`.

What you should see

- Delhi → Chennai = **1767.7 km**. The traveller's 10-minute hop implies **10,606 km/h**, so `IMPOSSIBLE TRAVEL` (limit 900).
- Account 1: average **5,980.49**, standard deviation **12,765.36**. Txn 12052 (₹95,000) has **z = 6.97**.
- The only 00:00-04:59 transaction on account 1 is **12043**.

Concepts you must be able to explain (viva)

- **Aggregate functions skip NULL.** …except `COUNT(*)`.
- **z-score.** `(x - mean) / standard deviation`: how many deviations from this account's normal. > 3 is unusual **if the data is bell-shaped**.
- **STDDEV_SAMP vs STDDEV_POP.** Sample (÷ n-1) when the rows are a sample of behaviour; population (÷ n) when they are everything. We use sample.
- **Haversine.** Great-circle distance on a sphere (radius 6371 km) from two latitude/longitude pairs.
- **User variables.** `SELECT … INTO @x` carries a value to the next statement. Session-scoped; Week 10 replaces them with `DECLARE`d variables.

More questions with full answers: `week-11-final-polish/docs/viva_questions.md`. Syntax for the closed-book lab: `docs/LAB_EXAM_CHEATSHEET.md`.

Handout step covered

Handout step 8 (advanced SQL: joins and subqueries ahead; functions underpin all rules).

Results (captured from the running database)

Evidence: Rule 1 by hand: the 95,000 payment

Script: `week-05-functions/queries/05_rule1_high_amount.sql`

```
+------------------+-----------------+
| account1_average | account1_stddev |
+------------------+-----------------+
|          5980.49 |        12765.36 |
+------------------+-----------------+
+--------+----------+---------+-------------------+
| txn_id | amount   | z_score | reason            |
+--------+----------+---------+-------------------+
|  12052 | 95000.00 |    6.97 | High amount (z=7) |
+--------+----------+---------+-------------------+
```

Evidence: Rule 5 by hand: Delhi to Chennai in 10 minutes

Script: `week-05-functions/queries/07_rule5_impossible_travel_by_hand.sql`

```
+-------------+---------+-------------------+-------------------+
| distance_km | minutes | implied_speed_kmh | verdict           |
+-------------+---------+-------------------+-------------------+
|      1767.7 |      10 |             10606 | IMPOSSIBLE TRAVEL |
+-------------+---------+-------------------+-------------------+
```

Week 6 — Operators: arithmetic, comparison, logical, special, set

Lab: Handout lab list **#6 — Operators** (ARITHMETIC, LOGICAL, COMPARISON, SPECIAL, SET) · lab manual "Implementation of different types of operators" **Theory lectures this lines up with:** 12–17 (operators)

Every operator family on real data, ending in a deliberately naive "first-cut fraud screen" built from fixed thresholds, which shows *why* the later weeks compare each account with its own history.

What was added

File	What it is
`queries/01_arithmetic_operators.sql`	`+ - * / DIV %`, precedence, division by zero.
`queries/02_comparison_operators.sql`	`= <> < > <= >= <=>`; the `= NULL` trap.
`queries/03_logical_operators.sql`	AND, OR, NOT, XOR and why brackets matter (1,300 rows vs 148).
`queries/04_special_operators.sql`	BETWEEN, IN, NOT IN, LIKE (`%` and `_`), IS NULL, REGEXP.
`queries/05_set_operators.sql`	UNION, UNION ALL, INTERSECT, EXCEPT on lists of accounts.
`queries/06_first_cut_fraud_screen.sql`	A screen from fixed rules of thumb: **216 transactions caught, only 27 planted**.

Files in `sql/` are **loaded by the build** (`scripts/build.sh`). Files in `queries/`, `demo/` and `lab-practice/` are run by hand and are checked by `scripts/check.sh`.

What you should see

- `amount > 5000 AND (status='failed' OR status='pending')` → **148** rows; the same without brackets → **1,300**.
- Accounts with a payment over 50,000: 8. Of those, **only account 20** also made a large night payment (INTERSECT); the other **7** are EXCEPT.
- The first-cut screen returns **216** rows. Most are innocent: the cost of fixed thresholds.

Concepts you must be able to explain (viva)

- **Precedence.** `NOT` > `AND` > `OR`; `* /` before `+ -`. Brackets remove doubt.
- **NULL logic.** Any comparison with NULL is *unknown*. Use `IS NULL` or `<=>`.
- **BETWEEN.** Inclusive at both ends.
- **LIKE.** `%` = any run of characters, `_` = exactly one.
- **Set operators.** Same number of columns, compatible types. UNION removes duplicates; UNION ALL keeps them. `EXCEPT` = Oracle's `MINUS` (MySQL 8.0.31+).
- **False positives.** Fixed thresholds treat a rich customer's normal payment as fraud. Rule 1 fixes that with a per-account baseline.

More questions with full answers: `week-11-final-polish/docs/viva_questions.md`. Syntax for the closed-book lab: `docs/LAB_EXAM_CHEATSHEET.md`.

Handout step covered

Handout step 8 (advanced SQL).

Results (captured from the running database)

Evidence: Set operators

Script: `week-06-operators/queries/05_set_operators.sql`

```
+------------+
| account_id |
+------------+
|          1 |
|         20 |
|        117 |
|        158 |
|        165 |
+------------+
+--------+
| a_or_b |
+--------+
|     31 |
+--------+
+----------------+
| rows_union_all |
+----------------+
|             32 |
+----------------+
+------------+
| account_id |
+------------+
|         20 |
+------------+
+-------------------+
| big_but_not_night |
+-------------------+
|                 7 |
+-------------------+
+-----------------+---------+------------+
| flagged_because | txn_id | account_id |
+-----------------+---------+------------+
| small hours     |  12043 |          1 |
| big amount      |  12052 |          1 |
+-----------------+---------+------------+
```

Evidence: First-cut fraud screen

Script: `week-06-operators/queries/06_first_cut_fraud_screen.sql`

```
+--------+------------+----------+------+---------+-----------------------------------------+
| txn_id | account_id | amount   | hr   | status  | why                                     |
+--------+------------+----------+------+---------+-----------------------------------------+
|  12052 |          1 | 95000.00 |   11 | success | very large amount, device 999 (never seen) |
|  12059 |        285 | 69888.23 |   18 | success | very large amount                       |
|  12053 |        117 | 66536.48 |   12 | success | very large amount                       |
|  12062 |        165 | 62629.96 |   21 | success | very large amount                       |
|  12061 |        158 | 61521.54 |   20 | success | very large amount                       |
|  12058 |        316 | 60943.44 |   17 | success | very large amount                       |
|  12067 |         20 | 59733.72 |    2 | success | very large amount, large night payment  |
|  12060 |        451 | 51180.91 |   19 | success | very large amount                       |
|  12065 |        358 | 34249.12 |    0 | success | large night payment                     |
|   2821 |         86 | 21238.33 |    4 | success | large night payment                     |
|   1337 |        302 | 15021.19 |    1 | success | large night payment                     |
|   5696 |        481 | 13373.93 |    4 | success | large night payment                     |
|    415 |        283 | 12474.93 |    0 | failed  | large night payment                     |
|   8207 |          7 | 12231.30 |    0 | success | large night payment                     |
|   2516 |        257 | 12205.23 |    4 | success | large night payment                     |
+--------+------------+----------+------+---------+-----------------------------------------+
+-------------+
| screened_in |
+-------------+
|         216 |
+-------------+
```
